

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль



Бие даалтын ажил

Параллел ба хуваарилагдсан тооцоолол (F.CSM306)
2025-2026 оны хичээлийн жилийн хавар

Бие даалтын ажлын нэр:
бөмбөлгөн эрэмблэлт

Хичээл заасан багш:

Багийн дугаар:

Гүйцэтгэсэн оюутан:

Дан холбоост жагсаалт дээрх

Н.АНХБАЯР

№1

Б.Энхтайван /B232270003/

З.Анхбаяр /B232270029/

М.Мөнхгэрэлт /B232270012/

Улаанбаатар хот

2026 он

Гарчиг

1	Хураангуй	4
2	Онолын ойлголтууд	5
2.1	Bubble Sort алгоритм	5
2.2	Odd-Even Transposition Sort	6
2.3	Дан холбоост жагсаалт	6
2.4	Параллельчлалын үндсэн санаа	7
2.5	OpenMP-ийн онолын үндэс	8
2.6	<code>std::thread</code> -ийн онолын үндэс	8
2.7	CUDA-ийн онолын үндэс	8
2.8	Хурдасгал(Speedup) ба Бүтээмж(Throughput)	9
2.9	Онолын дүгнэлт	10
3	Алгоритмын зохиомж	10
3.1	Зохиомжийн зураг	10
3.1.1	Өгөгдлийн бүтэц ба санах ой	10
3.1.2	Odd-Even phase-ийн ажиллагаа	11
3.1.3	CPU parallel архитектур	12
3.1.4	CUDA архитектур	12
3.2	Псевдо код	13
3.2.1	Sequential хувилбар	13
3.2.2	OpenMP хувилбар	13
3.2.3	<code>std::thread</code> хувилбар	14
3.2.4	CUDA хувилбар	14
3.3	Зангилаа, санах ой, кэш, регистрийн түвшний тайлбар	15
4	Үр дүн харьцуулсан жишээ	16
4.1	Хэмжилт	16
4.2	Корреляцийн диаграмм	17
4.3	Харьцуулалтын диаграмм	20

5	Дүгнэлт	21
5.1	Холбоосууд	21

№	Үнэлгээний үзүүлэлт	Дэлгэрэнгүй шалгуур	Оноо	Шалгах
1	Програм хангамжийн хэрэгжүүлэлт		6	
1.1	Sequential & OpenMP хувилбар	Алгоритмыг OpenMP ашиглан зөв parallel болгосон, үр дүн зөв гардаг байх.	1.5	✓
1.2	CUDA хувилбар	CUDA Kernel, Thread block, Grid бүтэц болон санах ойн дамжуулалтыг зөв кодлосон байх.	1.5	✓
1.3	Multi Thread хувилбар (std::thread)	C++ стандарт сан ашиглан Thread-ийн удирдлага болон синхрончлолыг хангаж multithreading хувилбарыг бичих.	1.5	✓
1.4	Кодын соёл	Програмын эх код дотор гол хэсгүүдийг тайлбарласан байх. Комментлогдсон, цэгцтэй код.	1.5	✓
2	Алгоритмын зохиомж ба онол		3.5	
2.1	Зохиомжийн зураг	Параллел алгоритмын ажиллагааны зарчмыг харуулсан блок схем, архитектурын зураглал.	1.5	□
2.2	Онолын тайлбар	Зангилаа, санах ой, кэш, регистрийн түвшний ажиллагааг онолын хувьд тайлбарласан байх.	1	✓
2.3	Псевдо код	Шинэ алгоритмуудын гол функцуудыг псевдо кодоор илэрхийлж, тайлбарлах.	1	✓
3	Шинжилгээ ба үр дүн		4.5	
3.1	Хэмжигдэхүүн	SpeedUp, Execution time, Data transfer time зэрэг үзүүлэлтийг өгөгдлийн өөр өөр хэмжээ дээр гаргах.	1.5	□

№	Үнэлгээний үзүүлэлт	Дэлгэрэнгүй шалгуур	Оноо	Шалгах
3.2	Харьцуулсан график	Алгоритмуудыг хооронд нь харьцуулсан график, диаграммууд.	1.5	✓
3.3	Дүгнэлт	Шинжилгээнд үндэслэн ямар алгоритм, ямар тохиолдолд үр дүнтэй байгааг нотолж бичих.	1.5	✓
4	Тайлангийн стандарт		2	
4.1	Тайлангийн бүтэц	Хураангуй, Оршил, Онол, Зохиомж, Үр дүн, Дүгнэлт гэсэн эрдэм шинжилгээний бүтэцтэй байх.	1	✓
4.2	Latex ба Ишлэл	Тайланг Latex дээр бэлтгэсэн бөгөөд болон ашигласан материалыг IEEE/APA стандартаар иш татсан байх.	1	✓
НИЙТ			16	

1. Хураангуй

Энэхүү ажлын хүрээнд дан холбоост жагсаалт дээр Odd-Even Bubble Sort алгоритмыг дөрвөн хувилбараар хэрэгжүүлж, гүйцэтгэлийг харьцуулсан. Үүнд дараалсан буюу sequential хувилбар, C++ стандарт сангийн `std::thread` ашигласан multithread хувилбар, OpenMP хувилбар, мөн GPU дээр CUDA ашигласан parallel хувилбарууд орно.

Odd-Even Bubble Sort нь Bubble Sort алгоритмын parallel хэлбэр бөгөөд нэг phase буюу шатлал дотор давхцахгүй хөрш элементүүдийг зэрэг харьцуулж солих боломжтой. Sequential хувилбар нь нэг урсгал ашиглан үндсэн суурь хэмжилт хийсэн. OpenMP хувилбарт `parallel for` ашиглан even болон odd шатлалтын хосуудыг зэрэг ажиллуулсан бол `std::thread` хувилбарт thread үүсгэх, ажил хуваарилах, phase буюу шатлалт хооронд synchronization хийх хэсгийг гараар хэрэгжүүлсэн. CUDA хувилбарт linked list-ийн утгуудыг array хэлбэрт хөрвүүлж GPU memory руу хуулан, phase бүр дээр CUDA

kernel ажиллуулсан.

Туршилтаар тооцооллын хугацаа (computation time), хэрэгжилтийн хугацаа (execution time), бүтээмж (throughput), хурдасгал (speedup), өгөгдөл дамжууллын хугацаа (data transfer time) болон эрэмбэлэгдсэн (sorted) эсэхийг хэмжсэн. Үр дүнгээс харахад параллель хувилбарууд нь зарим нөхцөлд гүйцэтгэлийг сайжруулсан боловч Bubble Sort алгоритмын $O(n^2)$ төвөгтэй байдал, синхрончлолын нэмэлт зардал, дан холбоост жагсаалтын кэш локалчлал муу байгаа зэрэг нь хурдасгалд нөлөөлсөн. CUDA хувилбарт олон элемент дээр зэрэг ажиллах боломжтой боловч CPU–GPU хооронд өгөгдөл дамжуулах нэмэлт зардал гарсан.

2. Онолын ойлголтууд

2.1. Bubble Sort алгоритм

Bubble Sort нь хөрш хоёр элементийг дараалан харьцуулж, буруу дараалалтай байвал байрыг нь сольдог энгийн эрэмбэлэх алгоритм юм. Нэг бүтэн давталтын дараа хамгийн их утга жагсаалтын төгсгөл рүү шилждэг. Энэ үйлдэл бүх элемент зөв дараалалд орох хүртэл давтагдана.

Bubble Sort-ийн үндсэн давуу тал нь ойлгоход хялбар, хэрэгжүүлэхэд энгийн байдаг. Харин сул тал нь том хэмжээний өгөгдөл дээр маш удаан ажилладаг. Учир нь алгоритм хөрш элементүүдийг олон удаа харьцуулдаг. [1], [2]

Хугацааны төвөгтэй байдал:

$$O(n^2)$$

Энд n нь эрэмбэлэх элементийн тоо юм. Хамгийн муу тохиолдолд нийт харьцуулалтын тоо ойролцоогоор:

$$\frac{n(n-1)}{2}$$

байна.[1]. Иймээс өгөгдлийн хэмжээ 10 дахин өсөхөд хийх ажлын хэмжээ ойролцоогоор 100 дахин өсдөг.

2.2. Odd-Even Transposition Sort

Odd-Even Transposition Sort нь Bubble Sort алгоритмыг parallel байдлаар хэрэгжүүлэхэд тохиромжтой хэлбэр юм[3]. Энэ алгоритм нь эрэмбэлэлтийг even phase болон odd phase гэж хоёр хэсэгт хуваадаг.

Even phase үед дараах хосуудыг харьцуулна:

$$(0, 1), (2, 3), (4, 5), \dots$$

Odd phase үед дараах хосуудыг харьцуулна:

$$(1, 2), (3, 4), (5, 6), \dots$$

Нэг phase доторх хосууд хоорондоо давхцахгүй. Жишээлбэл, even phase-ийн үед $(0, 1)$ болон $(2, 3)$ хосууд өөр өөр элементүүд дээр ажиллаж байгаа тул эдгээрийг зэрэг ажиллуулах боломжтой. Энэ шинж чанар нь алгоритмыг OpenMP, `std::thread`, CUDA зэрэг parallel технологиор хэрэгжүүлэх үндэс болсон.

Гэхдээ even phase болон odd phase нь хоорондоо хамааралтай. Even phase бүрэн дуусаагүй байхад odd phase эхэлбэл нэг элемент дээр хоёр өөр урсгал зэрэг хандах боломжтой. Энэ нь race condition үүсгэж буруу үр дүнд хүргэж болзошгүй. Тиймээс phase хооронд synchronization буюу barrier(хаалт) шаардлагатай.

2.3. Дан холбоост жагсаалт

Энэ ажилд өгөгдлийг дан холбоост жагсаалт хэлбэрээр хадгалсан. Дан холбоост жагсаалтын зангилаа бүр өгөгдөл болон дараагийн зангилааны хаягийг хадгална.

$$[data|next] \rightarrow [data|next] \rightarrow [data|null]$$

Array-тай харьцуулахад linked list нь санах ойд дараалсан байрлах албагүй. Тиймээс индексээр шууд хандах боломжгүй бөгөөд дараагийн node руу pointer ашиглан шилждэг. Энэ нь кэш локалчлалыг муу болгодог. Өөрөөр хэлбэл CPU нэг зангилааг унших үед дараагийн зангилаа нь ойролцоох санах ойд байх баталгаа байхгүй.

Иймээс parallel хувилбаруудад дан холбоост жагсаалт дээр шууд индекс ашиглах боломжгүй асуудалтай тулгарсан. Үүнийг шийдэхийн тулд CPU parallel хувилбаруудад дан холбоост жагсаалтын зангилаануудын хаягийг `vector<Node*>` дотор хадгалсан. Ингэснээр зангилааны өгөгдлийг хуулбарлахгүйгээр индексээр хандаж, тухайн зангилааны `data` утгыг сольж чадсан.

CUDA хувилбарт дан холбоост жагсаалтын утгуудыг массив хэлбэрт хөрвүүлсэн. Учир нь GPU дээр заагчаар холбогдсон дан холбоост жагсаалт боловсруулах нь үр ашиг багатай, харин массив дээр олон урсгал зэрэг ажиллахад илүү тохиромжтой.

2.4. Параллельчлалын үндсэн санаа

Параллель програмчлалын гол санаа нь том ажлыг жижиг хэсгүүдэд хувааж, олон процессорын цөм эсвэл олон урсгал дээр зэрэг ажиллуулах явдал юм[4]. Энэ ажлын хувьд нэг phase доторх хөрш элементүүдийн хосууд бие биеэсээ хамааралгүй тул зэрэг харьцуулж сольж болно.

Жишээлбэл:

$$(0, 1), (2, 3), (4, 5)$$

гэсэн хосууд дээр зэрэг ажиллахад нэг элемент хоёр хосод давхар оролцохгүй. Харин дараагийн odd phase дээр:

$$(1, 2), (3, 4), (5, 6)$$

гэсэн хосууд шалгагдана. Энэ шилжилтийн үед өмнөх phase бүрэн дууссан байх ёстой.

Параллельчлалын үр ашиг нь дараах хүчин зүйлсээс хамаарна:

- урсгал үүсгэх болон удирдах зардал,
- синхрончлол буюу barrier(хаалт)-ийн зардал,
- өгөгдөл санах ойд хэрхэн байрласан байдал,
- кэш ашиглалт,

- алгоритмын нийт төвөгшил.

2.5. OpenMP-ийн онолын үндэс

OpenMP нь C/C++ програмд parallel region болон parallel loop үүсгэхэд ашиглагддаг API(интерфейс) юм [5]. OpenMP ашигласнаар программист урсгал бүрийг гараар үүсгэх шаардлагагүй. Харин хөрвүүлэх директив буюу `#pragma omp parallel for` ашиглан loop-ийн давталтуудыг олон урсгалд хуваарилдаг [5].

OpenMP хувилбарт odd-even phase бүрийн хосуудыг parallel loop ашиглан ажиллуулсан. OpenMP-ийн `parallel for` байгуулагчийн төгсгөлд далд barrier(хаалт) байдаг. Энэ нь бүх урсгал тухайн loop-ийг дуусгасны дараа дараагийн phase эхлэхийг баталгаажуулдаг.

Иймээс OpenMP хувилбарт тусдаа barrier(хаалт) бичих шаардлагагүй. синхрончлолыг OpenMP runtime өөрөө удирдана.

2.6. std::thread-ийн онолын үндэс

C++ стандарт сангийн `std::thread` нь урсгалыг программист өөрөө үүсгэж, удирдах боломж олгодог [6]. OpenMP-ээс ялгаатай нь `std::thread` ашиглах үед ажлыг урсгал бүрт хэрхэн хуваах, урсгалуудыг хэрхэн синхрончлох, хэзээ `join` хийх зэргийг гараар шийдэх шаардлагатай.

Multithread хувилбарт урсгал бүр odd-even phase дотор өөрт ноогдсон хосуудыг боловсруулдаг. Phase хооронд урсгалуудыг зэрэгцүүлэхийн тулд тусдаа `ReusableBarrier` ашигласан. Barrier(хаалт) нь бүх урсгал тухайн phase-ээ дуусгах хүртэл дараагийн phase эхлэхээс сэргийлдэг.

2.7. CUDA-ийн онолын үндэс

CUDA нь NVIDIA GPU дээр parallel програм бичихэд ашиглагддаг платформ юм [7]. CUDA програмд CPU талын кодыг host code, GPU дээр ажиллах кодыг device code гэж нэрлэдэг. GPU дээр ажиллах функцыг kernel гэж нэрлэх бөгөөд олон урсгал зэрэг ажиллуулдаг.

CUDA-ийн урсгал зохион байгуулалт нь дараах бүтэцтэй: [7]:

$$Grid \rightarrow Blocks \rightarrow Threads$$

Speedup ойлголт нь Amdahl-ийн хуулиар тайлбарлагддаг [8]. CUDA хувилбарт дан холбоост жагсаалтын өгөгдлийг массив болгон хувиргаж GPU memory руу хуулсан. Дараа нь odd эсвэл even phase бүр дээр CUDA kernel ажиллуулж, GPU thread бүр нэг хөрш хос шалгах байдлаар зохион байгуулсан.

CUDA хувилбарын гол давуу тал нь олон мянган урсгал ашиглан их хэмжээний өгөгдлийг зэрэг боловсруулах боломж юм. Харин сул тал нь CPU болон GPU-ийн хооронд өгөгдөл дамжуулах нэмэлт зардалтай. Иймээс CUDA хувилбар дээр өгөгдөл дамжууллын хугацаа (data transfer time) болон дамжуулсан өгөгдлийн хэмжээ (amount of data transferred) гэсэн үзүүлэлтүүдийг тусад нь авч үзэх шаардлагатай.

2.8. Хурдасгал(Speedup) ба Бүтээмж(Throughput)

Параллель алгоритмын үр ашгийг үнэлэхэд хурдасгал(speedup) болон бүтээмж(throughput) гэсэн үзүүлэлтүүдийг ашигласан.

Хурдасгал нь sequential хувилбартай харьцуулахад parallel хувилбар хэд дахин хурдан ажилласныг харуулна.

$$SpeedUp = \frac{T_{sequential}}{T_{parallel}}$$

Бүтээмж(Throughput) нь нэг секундэд хэдэн элемент боловсруулж байгааг илэрхийлнэ.

$$Throughput = \frac{n}{ExecutionTime}$$

Энд *ExecutionTime*-ийг секундээр илэрхийлнэ.

Эдгээр үзүүлэлтүүд нь зөвхөн урсгалын тооноос хамаарахгүй. Синхрончлолын нэмэлт зардал, санах ойн хандалтын загвар(memory access pattern), өгөгдөл дамжууллын хугацаа(data transfer time), кэш локалчлал зэрэг хүчин зүйлс мөн их нөлөөлдөг.

2.9. Онолын дүгнэлт

Онолын хувьд Odd-Even Bubble Sort нь Bubble Sort-ийг parallel хэлбэрээр хэрэгжүүлэх боломж олгодог. Нэг phase доторх хосууд давхцахгүй тул зэрэг ажиллуулах боломжтой боловч phase хооронд barrier(хаалт) шаардлагатай. OpenMP энэ barrier(хаалт)-ийг далд байдлаар хийдэг бол `std::thread` хувилбарт тусдаа barrier(хаалт) хэрэгжүүлэх шаардлагатай. CUDA хувилбарт GPU-ийн олон thread ашиглаж болох боловч дан холбоост жагсаалтыг массив болгон хөрвүүлэх, CPU-GPU хооронд өгөгдөл дамжуулах нэмэлт зардал гардаг.

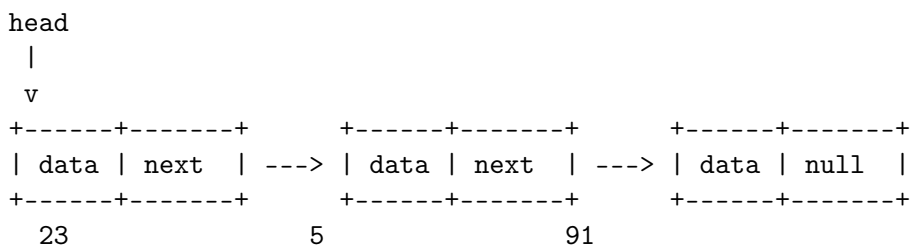
Иймээс онолын үндсэн санаа нь Bubble Sort-ийн $O(n^2)$ төвөгтэй байдалыг өөрчлөхгүйгээр, phase доторх хамааралгүй ажлуудыг parallel байдлаар гүйцэтгэж гүйцэтгэлийг харьцуулах юм.

3. Алгоритмын зохиомж

3.1. Зохиомжийн зураг

3.1.1. Өгөгдлийн бүтэц ба санах ой

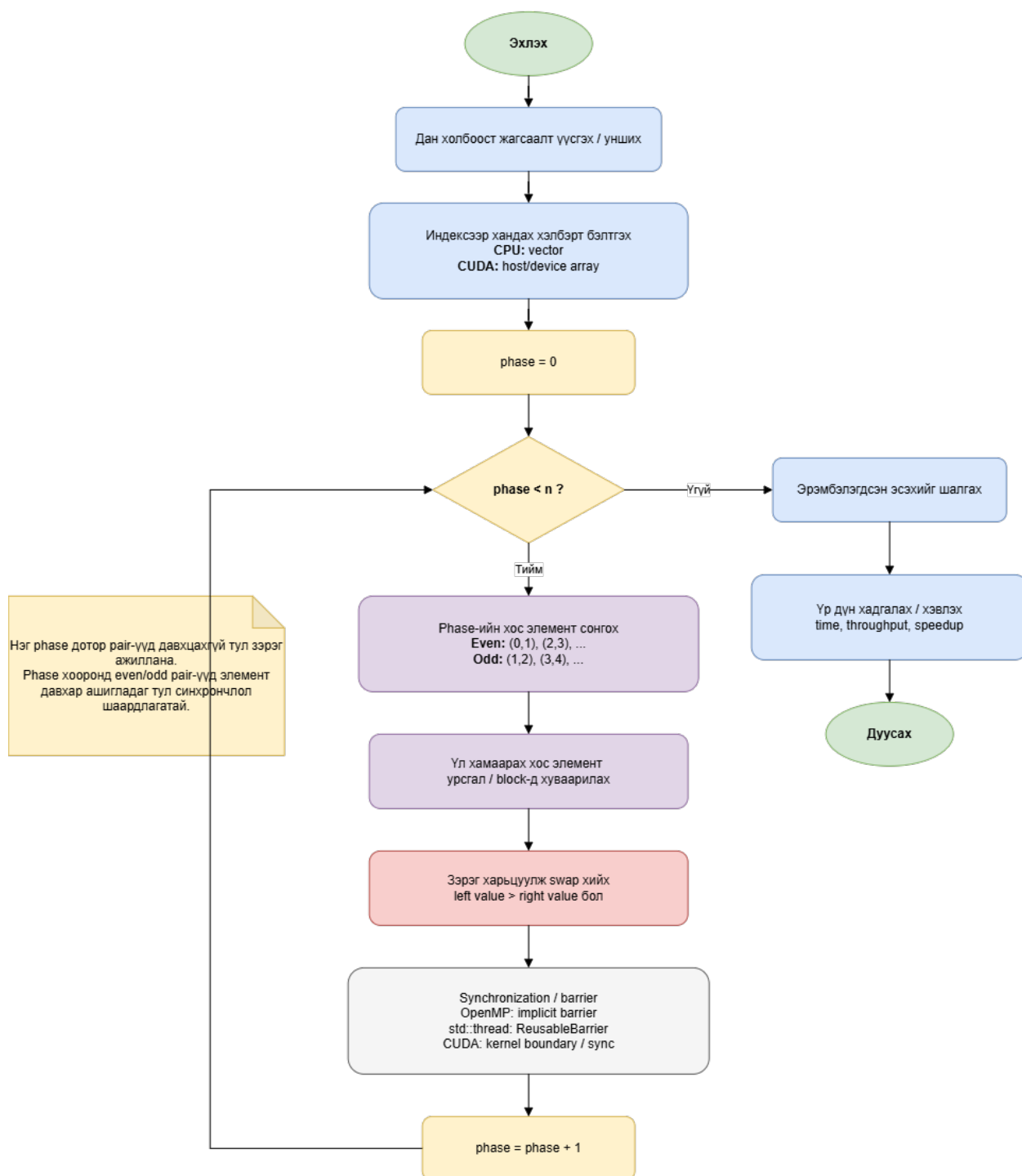
Дан холбоост жагсаалт



Дан холбоост жагсаалтын зангилаа бүр `data` болон `next` гэсэн хоёр хэсэгтэй. Зангилаанууд hear(овоо) санах ойд заавал дараалж байрлахгүй. Иймээс массив шиг шууд индексээр хандах боломжгүй, заагч дагаж дараагийн зангилаа руу шилжинэ.

3.1.2. Odd-Even phase-ийн ажиллагаа

Odd-Even Bubble Sort параллел алгоритмын блок схем



Зураг 3.1: Odd-Even Bubble Sort параллел алгоритмын блок диаграм

Index: 0 1 2 3 4 5

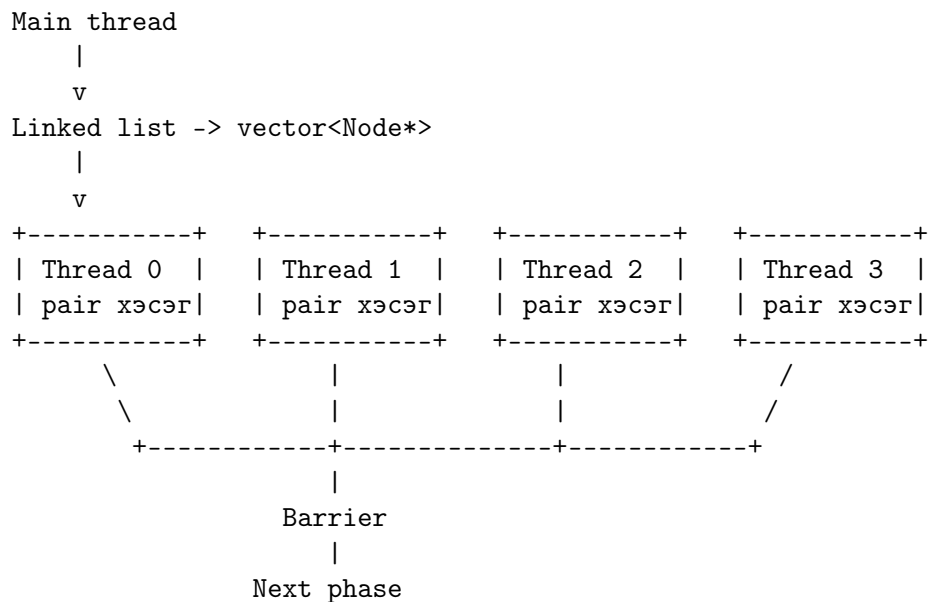
Data: [8] [3] [7] [1] [5] [2]

Even phase:
 (0,1) (2,3) (4,5)

Odd phase:
 (1,2) (3,4)

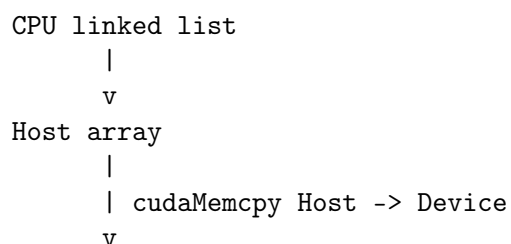
Нэг phase доторх хосууд давхцахгүй тул зэрэг ажиллуулж болно. Харин even phase дуусаагүй байхад odd phase эхэлбэл нэг элемент дээр хоёр урсгал зэрэг хандах эрсдэлтэй. Тиймээс phase хооронд barrier(хаалт) хэрэгтэй.

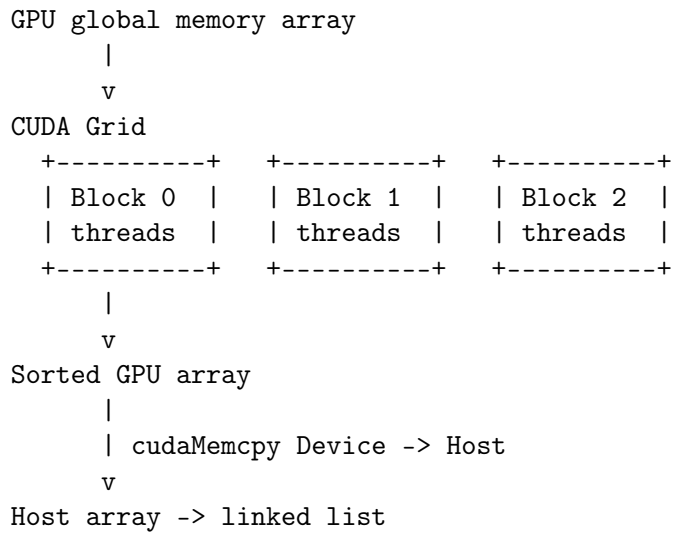
3.1.3. CPU parallel архитектур



OpenMP хувилбарт энэ barrier(хаалт)-ийг OpenMP runtime далд байдлаар хийдэг. `std::thread` хувилбарт barrier(хаалт)-ийг `ReusableBarrier` нэртэй тусдаа синхрончлолын класс ашиглан гараар хэрэгжүүлсэн.

3.1.4. CUDA архитектур





CUDA дээр дан холбоост жагсаалтыг шууд заагч хэлбэрээр боловсруулах нь үр ашиг багатай. Тиймээс эхлээд **data** утгуудыг массив болгож GPU memory руу хуулсан. GPU-ны урсгал бүр нэг зэргэлдээ хосыг шалгаж солидог.

3.2. Псевдо код

3.2.1. Sequential хувилбар

```

SequentialBubbleSort(head):
    repeat:
        current = head
        swapped = false

        while current.next exists:
            if current.data > current.next.data:
                swap(current.data, current.next.data)
                swapped = true

            current = current.next

        if swapped == false:
            break

```

Энэ нь baseline хувилбар. Нэг урсгал дан холбоост жагсаалтаар дараалан явж зэргэлдээ зангилаануудыг харьцуулна. Солигдохгүй бол жагсаалт эрэмбэлэгдсэн гэж үзнэ.

3.2.2. OpenMP хувилбар

```

OpenMPBubbleSort(head, n, threadCount):

```

```

nodes = linked list-ийн node хаягуудыг vector-д хадгална

repeat phase from 0 to n - 1:
    start = phase % 2

    parallel for i = start; i < n - 1; i += 2:
        if nodes[i].data > nodes[i + 1].data:
            swap(nodes[i].data, nodes[i + 1].data)

    implicit barrier

```

OpenMP loop-ийн давталтуудыг урсгалуудад автоматаар хуваарилна. `parallel for`-ийн төгсгөлд далд `barrier`(хаалт) байдаг тул дараагийн phase зөв дарааллаар эхэлнэ.

3.2.3. `std::thread` хувилбар

```

ThreadBubbleSort(head, threadCount):
    nodes = linked list-ийн node хаягууд
    barrier = ReusableBarrier(threadCount)
    swapped = false
    done = false

    worker(threadId):
        repeat round:
            өөрт ноогдсон even pair-үүдийг шалгаж swap хий
            barrier.wait()

            өөрт ноогдсон odd pair-үүдийг шалгаж swap хий
            barrier.wait():
                done = swapped == false
                swapped = false

        if done:
            break

    бүх worker thread-үүдийг эхлүүл
    бүх thread-үүдийг join хий

```

Энэ хувилбарт урсгалын удирдлага гараар хийгдэнэ. Урсгал бүр өөрийн хосын хэсгийг боловсруулна. `Barrier`(хаалт) нь бүх урсгал нэг phase-ээ дуусгаснаар дараагийн phase эхлэхийг баталгаажуулна.

3.2.4. CUDA хувилбар

```

CudaOddEvenSort(head, n):

```

```

hostArray = linked list-ийн data утгууд
deviceArray = GPU memory

copy hostArray -> deviceArray

repeat phase from 0 to n - 1:
    launch oddEvenKernel(deviceArray, n, phase % 2)

synchronize GPU

copy deviceArray -> hostArray
hostArray-г linked list рүү буцааж бич

```

CUDA kernel:

```

oddEvenKernel(array, n, phase):
    threadId = blockIdx.x * blockDim.x + threadIdx.x
    index = phase + 2 * threadId

    if index + 1 < n:
        if array[index] > array[index + 1]:
            swap(array[index], array[index + 1])

```

CUDA урсгал бүр нэг хос шалгана. Phase бүр дээр kernel ажиллана. CUDA хувилбарын давуу тал нь олон урсгал зэрэг ажиллах боломжтой, харин сул тал нь CPU-GPU хооронд өгөгдөл дамжуулах нэмэлт зардалтай.

3.3. Зангилаа, санах ой, кэш, регистрийн түвшний тайлбар

Зангилааны түвшинд алгоритмд зангилаанууд заагчийн холбоосыг өөрчлөхгүй, зөвхөн data утгыг сольдог. Энэ нь дан холбоост жагсаалтын бүтэц эвдрэх эрсдэлийг багасгана.

Санах ойн түвшинд дан холбоост жагсаалтын зангилаанууд heap(овоо) дээр тархсан байрлалтай байж болно. Тиймээс CPU кэшд дараалсан байдлаар орох магадлал массивтай харьцуулахад бага.

Кэшийн түвшинд `vector<Node*>` ашигласнаар зангилааны хаягууд дараалсан санах ой дээр байрлана. Гэхдээ бодит data нь зангилаан дээр байгаа тул заагч dereference хийх зардал хэвээр үлдэнэ.

Регистрийн түвшинд харьцуулалт хийх үед хоёр зэргэлдээ элементийн утга түр регистрт ачаалагдаж, `if a > b` шалгалт хийгдэнэ. Солих шаардлагатай бол регистр дээрх түр утгуудыг ашиглаад санах ойруу буцааж бичнэ.

4. Үр дүн харьцуулсан жишээ

4.1. Хэмжилт

	Type	Size	ThreadsPerBlock	Blocks_mean	ComputationTimeMs_mean	ExecutionTimeMs_mean	DataTransferTimeMs_mean	Throughput_mean
0	Cuda Odd-Even	10000	64	79.0	73.851	74.487	0.081	134681.380
1	Cuda Odd-Even	10000	128	40.0	81.258	82.038	0.088	123919.032
2	Cuda Odd-Even	10000	256	20.0	77.944	78.714	0.077	134123.402
3	Cuda Odd-Even	10000	512	10.0	79.074	79.704	0.090	129294.788
4	Cuda Odd-Even	100000	64	782.0	754.241	756.195	0.168	132201.655
5	Cuda Odd-Even	100000	128	391.0	828.870	830.678	0.183	120611.742
6	Cuda Odd-Even	100000	256	196.0	804.132	805.941	0.164	124810.360
7	Cuda Odd-Even	100000	512	98.0	764.857	766.801	0.220	130509.578
8	Cuda Odd-Even	200000	64	1563.0	1634.630	1637.926	0.282	122091.330
9	Cuda Odd-Even	200000	128	782.0	1608.880	1611.847	0.258	124234.157
10	Cuda Odd-Even	200000	256	391.0	1571.505	1574.380	0.257	126994.207
11	Cuda Odd-Even	200000	512	196.0	1600.203	1603.848	0.373	125039.262

Зураг 4.1: Cuda хэмжилт

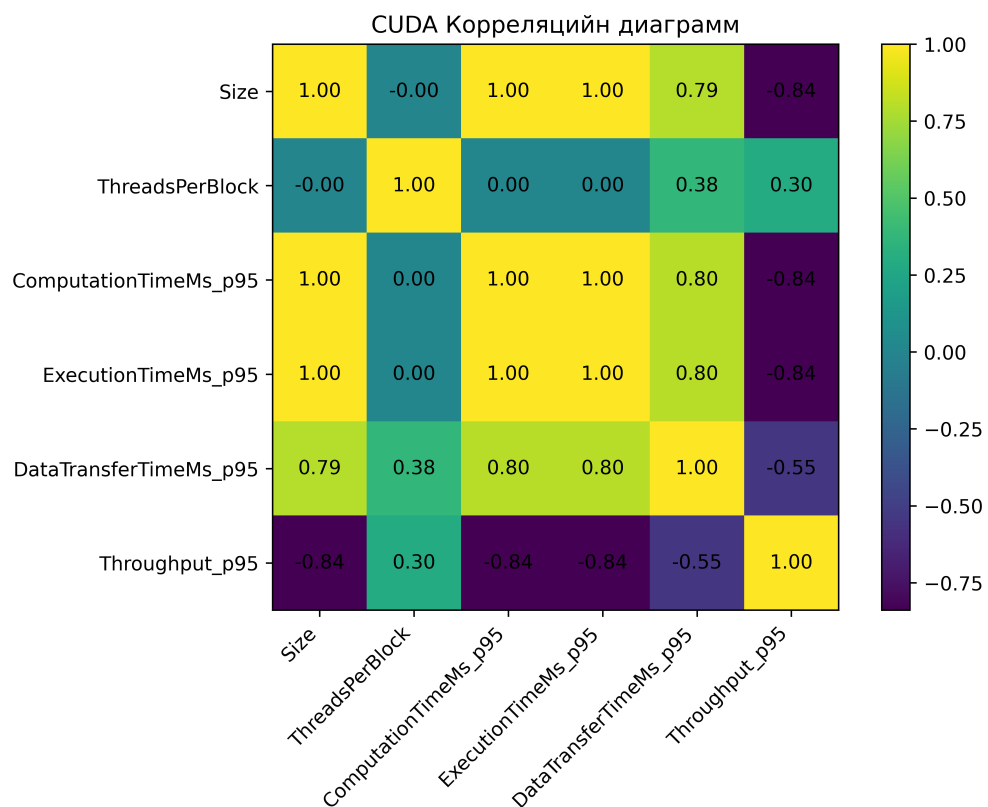
	Type	Size	Threads	ComputationTimeMs_mean	ExecutionTimeMs_mean	DataTransferTimeMs_mean	Throughput_mean	Speedup_mean	Sorted
0	Multithread	10000	1	173.936	173.936	0.0	57492.48	1.00	Yes
1	Multithread	10000	2	491.160	491.160	0.0	20359.95	0.35	Yes
2	Multithread	10000	4	543.586	543.586	0.0	18396.35	0.32	Yes
3	Multithread	10000	8	572.193	572.193	0.0	17476.62	0.30	Yes
4	Multithread	10000	16	771.355	771.355	0.0	12964.21	0.23	Yes
5	Multithread	100000	1	20608.966	20608.966	0.0	4852.26	1.00	Yes
6	Multithread	100000	2	51920.715	51920.715	0.0	1926.01	0.40	Yes
7	Multithread	100000	4	67368.503	67368.503	0.0	1484.37	0.31	Yes
8	Multithread	100000	8	75577.001	75577.001	0.0	1323.15	0.27	Yes
9	Multithread	100000	16	74907.139	74907.139	0.0	1334.99	0.28	Yes
10	Multithread	200000	1	103234.101	103234.101	0.0	1937.34	1.00	Yes
11	Multithread	200000	2	218363.889	218363.889	0.0	915.90	0.47	Yes
12	Multithread	200000	4	286643.345	286643.345	0.0	697.73	0.36	Yes
13	Multithread	200000	8	312194.298	312194.298	0.0	640.63	0.33	Yes
14	Multithread	200000	16	300498.437	300498.437	0.0	665.56	0.34	Yes

Зураг 4.2: Multithread хэмжилт

	Type	Size	Threads	ComputationTimeMs_mean	ExecutionTimeMs_mean	DataTransferTimeMs_mean	Throughput_mean	Speedup_mean	Sorted
0	OpenMP	10000	2	441.319000	441.393000	0.073333	23053.573333	0.423333	Yes
1	OpenMP	10000	4	342.537333	342.593667	0.056333	30030.166667	0.550000	Yes
2	OpenMP	10000	8	536.420000	536.475000	0.055000	18833.640000	0.343333	Yes
3	OpenMP	10000	16	1203.726333	1203.793000	0.066333	8579.230000	0.156667	Yes
4	OpenMP	100000	2	30938.380333	30939.122333	0.742333	3304.973333	0.946667	Yes
5	OpenMP	100000	4	23034.398333	23035.438000	1.039333	4517.846667	1.286667	Yes
6	OpenMP	100000	8	18637.523000	18638.591667	1.068333	5524.500000	1.580000	Yes
7	OpenMP	100000	16	16922.778000	16923.897667	1.120000	6726.826667	1.910000	Yes
8	OpenMP	200000	2	114581.745000	114583.124000	1.379000	1756.446667	1.090000	Yes
9	OpenMP	200000	4	82293.516333	82295.630667	2.114333	2436.103333	1.516667	Yes
10	OpenMP	200000	8	55070.344000	55072.590667	2.246667	3636.453333	2.260000	Yes
11	OpenMP	200000	16	39747.312667	39749.574000	2.261333	5040.693333	3.133333	Yes
12	Sequential	10000	1	182.946667	182.946667	0.000000	54771.393333	1.000000	Yes
13	Sequential	100000	1	28703.385333	28703.385333	0.000000	3488.543333	1.000000	Yes
14	Sequential	200000	1	124555.319333	124555.319333	0.000000	1607.283333	1.000000	Yes

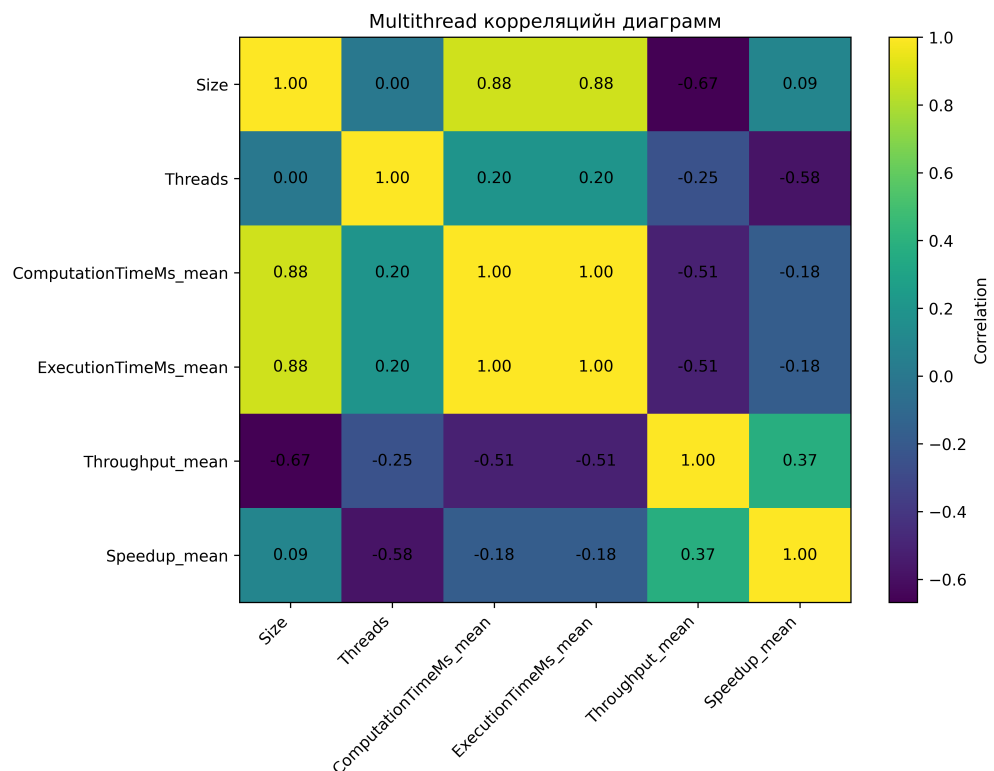
Зураг 4.3: OpenMP хэмжилт

4.2. Корреляцийн диаграмм



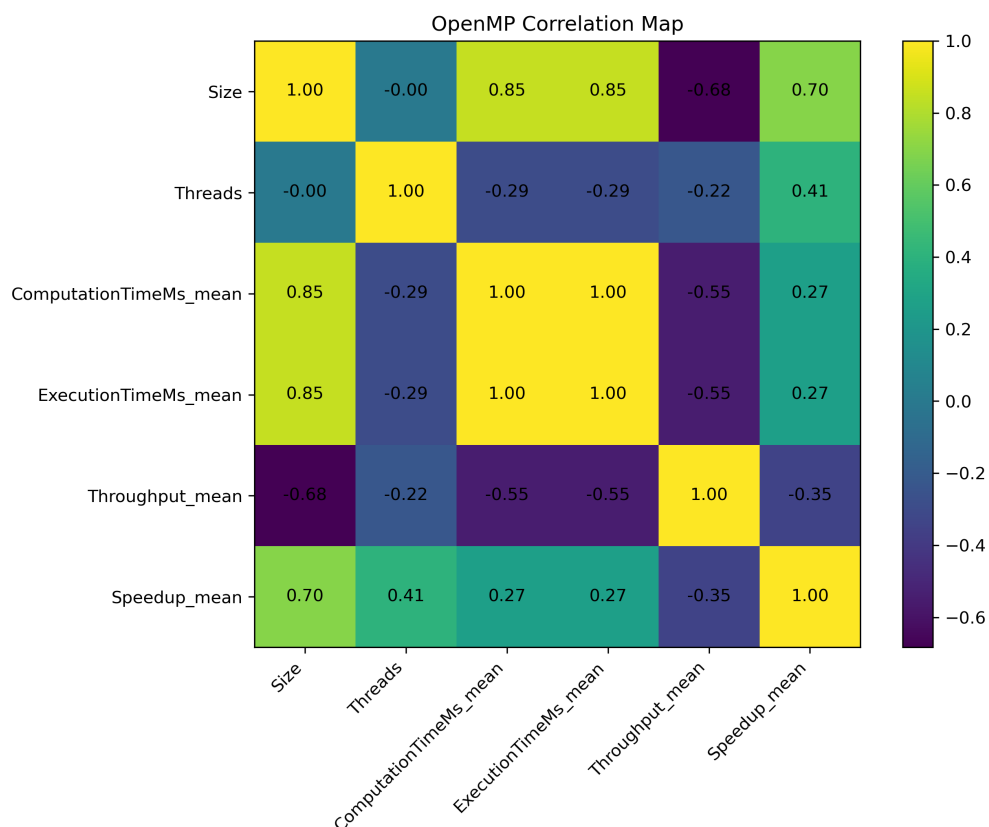
Зураг 4.4: CUDA корреляцийн диаграмм

CUDA odd-even sort алгоритмын p95 гүйцэтгэлийн хэмжилтээс харахад системийн performance-д хамгийн их нөлөөлж буй хүчин зүйл нь өгөгдлийн хэмжээ байна. Оролтын хэмжээ нэмэгдэхэд computation time, execution time, data transfer time өсөж, харин throughput буурч байна. Thread per block-ийн өөрчлөлт нь энэ туршилтад гүйцэтгэлийн хугацаанд хүчтэй нөлөө үзүүлээгүй тул алгоритмын bottleneck нь thread-ийн тооноос илүүтэй odd-even sort-ийн олон pass шаарддаг бүтэц болон өгөгдлийн хэмжээтэй холбоотой гэж үзэж болно.



Зураг 4.5: Multithread корреляцийн диаграмм

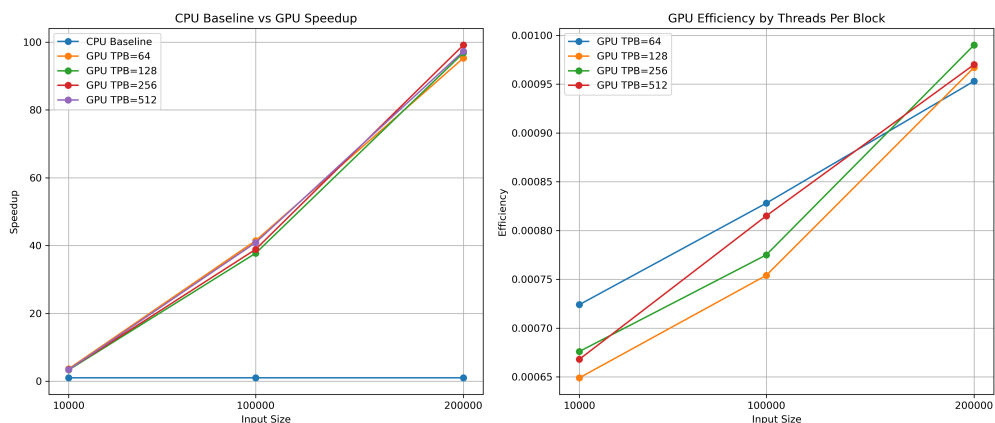
Multithread odd-even sort алгоритмын гүйцэтгэлд хамгийн хүчтэй нөлөөлж буй хүчин зүйл нь өгөгдлийн хэмжээ байна. Харин thread-ийн тоог нэмэгдүүлэх нь энэ туршилтад performance-ийг тогтмол сайжруулаагүй бөгөөд speedup-д сөрөг нөлөө үзүүлэх хандлагатай байна. Иймээс multithread хувилбарын хувьд оновчтой thread-ийн тоог сонгох нь чухал бөгөөд хэт олон thread ашиглах нь synchronization болон scheduling overhead-оос болж үр ашиг бууруулах боломжтой.



Зураг 4.6: OpenMP корреляцийн диаграмм

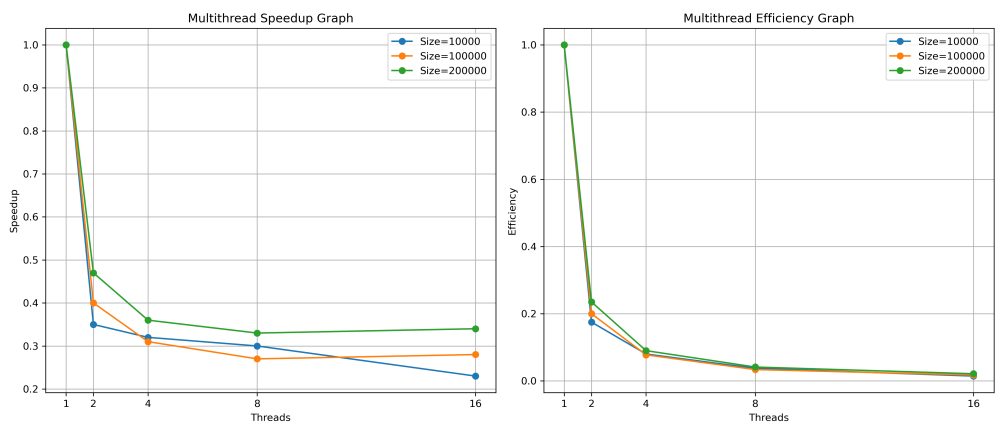
OpenMP хувилбарын гүйцэтгэлд хамгийн хүчтэй нөлөөлж буй хүчин зүйл нь өгөгдлийн хэмжээ байна. Thread-ийн тоо нэмэгдэхэд speedup өсөх хандлагатай боловч ажиллах хугацаанд үзүүлэх нөлөө хязгаарлагдмал байна. Иймээс OpenMP odd-even sort алгоритм нь том хэмжээний өгөгдөл дээр илүү үр ашигтай боловч performance-ийн гол bottleneck нь computation time хэвээр байна.

4.3. Харьцуулалтын диаграмм

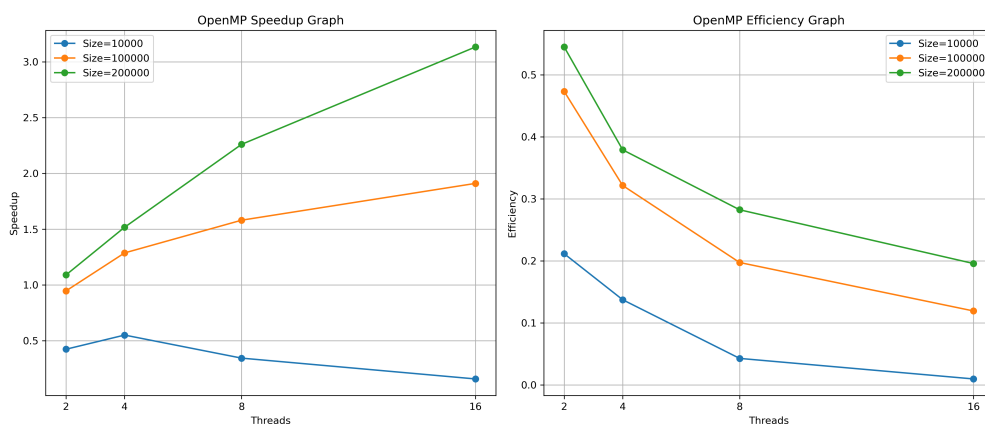


Зураг 4.7: Cuda speedup болон efficiency-г sequential-тай харьцуулсан байдал

Speedup-ийн ялгаа бага байсан ч efficiency-ийн хувьд 256 threads per block тохиргоо илүү тогтвортой, том өгөгдөл дээр илүү ашигтай үр дүн үзүүлсэн. Тиймээс CUDA odd-even sort алгоритмын хувьд 200000-аас дээш өгөгдөл боловсруулах үед 256 threads per block тохиргоог сонгох нь илүү оновчтой байна.



Зураг 4.8: Multithread speedup болон efficiency-г харуулсан байдал



Зураг 4.9: OpenMP speedup болон efficiency-г харуулсан байдал

5. Дүгнэлт

Туршилтын үр дүнгээр CUDA GPU хэрэгжилт хамгийн өндөр speedup гаргасан бөгөөд ялангуяа 100,000-аас дээш хэмжээтэй өгөгдөл дээр хамгийн тохиромжтой байна. 200,000 хэмжээтэй өгөгдөл дээр GPU speedup 95–99 орчим хүрсэн нь CPU baseline болон бусад CPU-д суурилсан хэрэгжилтүүдээс олон дахин хурдан байгааг нотолж байна.

OpenMP хэрэгжилт нь CPU дээр ажиллах хувилбаруудаас хамгийн үр дүнтэй байна. Энэ нь 100,000 болон 200,000 хэмжээтэй өгөгдөл дээр thread-ийн тоо нэмэгдэхэд speedup өссөнөөр батлагдаж байна. Гэхдээ OpenMP-ийн efficiency thread-ийн тоо нэмэгдэхэд буурч байгаа тул thread-ийн тоог хязгааргүй нэмэх нь үр ашиггүй. Энэ туршилтад 200,000 хэмжээтэй өгөгдөл дээр 16 thread ашиглахад хамгийн сайн speedup гарсан.

Энгийн multithread хэрэгжилт нь хамгийн үр дүн муутай байна. Учир нь thread-ийн тоо нэмэгдэхэд speedup буурч, efficiency огцом багассан. Энэ нь thread synchronization, thread creation, memory access overhead зэрэг зардлууд алгоритмын бодит тооцооллоос их байгааг харуулж байна. Иймээс энэхүү хэрэгжилтийг маш цөөн тооны урсгал ашиглан хэрэглэх нь тохиромжтой.

5.1. Холбоосууд

Кодыг ажиллуулж буй жишээ бичлэг

Github холбоос

Ашигласан материал

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest **and** C. Stein, *Introduction to Algorithms*, 3 **edition**. MIT Press, 2009.
- [2] R. Sedgewick **and** K. Wayne, *Algorithms*, 4 **edition**. Addison-Wesley, 2011.
- [3] A. N. Habermann, ?Parallel Neighbor Sort (or the Glory of the Induction Principle),? *Carnegie-Mellon University Technical Report*, 1972.
- [4] P. Pacheco, *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011.
- [5] *OpenMP Application Programming Interface Version 5.2*, OpenMP Architecture Review Board, 2021. **url**: <https://www.openmp.org/specifications/>
- [6] *std::thread*, cppreference.com, 2024. **url**: <https://en.cppreference.com/w/cpp/thread/thread>
- [7] *CUDA C++ Programming Guide*, NVIDIA Corporation, 2024. **url**: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [8] G. M. Amdahl, ?Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,? *AFIPS Conference Proceedings*, **jourvol** 30, **pages** 483–485, 1967.